

SBT-DF203 — BASIC NETWORKING SKILLS FOR DIGITAL FORENSICS

LAB 3 — SYN FLOOD PATTERN INVESTIGATION USING TSHARK

Student Name: Oluyede Tobi Steven

Registration Number: [REGISTRATION NUMBER]

Course: SBT-DF203 — Basic Networking Skills for Digital Forensics

Lab: Lab 3 — SYN Flood Pattern Investigation Using TShark

Instructor: Zakariyya Dahuru Musa

Date: 11 September 2026

1. Executive Summary

This laboratory exercise investigated TCP SYN activity using TShark in an authorised digital-forensics training environment. The objective was to establish the characteristics of a normal TCP three-way handshake and compare them with a bounded training capture containing repeated TCP SYN packets.

The supplied training capture, `mySYNFloodCapture.pcap`, was examined using TShark. The analysis identified **four TCP packets**, all of which were initial SYN packets directed to TCP port 80. The source and destination address in every observed packet was `127.0.0.1`, indicating loopback traffic within the controlled training environment.

The four observed source ports were **62558, 19550, 63314, and 34942**. No SYN-ACK, ACK, or RST packets were present in the supplied capture. The four SYN packets occurred between `0.0000000000` and `0.002756569` seconds, giving an observed capture span of approximately **2.76 milliseconds**.

The packet pattern is consistent with incomplete TCP connection attempts and demonstrates the characteristics required for recognising SYN-related activity. However, the capture contains only four SYN packets and therefore does **not by itself prove that a SYN flood caused service denial or resource exhaustion**. Additional evidence such as server availability measurements, connection-backlog information, system resource statistics, Apache logs, or a longer traffic capture would be required to establish actual service impact.

2. Objectives

The objectives of this laboratory were to:

1. Establish a normal TCP three-way handshake baseline.
 2. Differentiate complete and incomplete TCP handshakes.
 3. Analyse an authorised bounded TCP SYN training capture.
 4. Filter SYN, SYN-ACK, ACK, and RST packets using TShark.
 5. Quantify the number of observed SYN packets.
 6. Identify unique TCP source ports.
 7. Analyse packet timing and construct a forensic timeline.
 8. Identify indicators associated with incomplete TCP handshakes.
 9. Explain the limitations of the evidence.
 10. Recommend appropriate detection and mitigation controls.
-

3. Laboratory Environment

The investigation was conducted within an authorised isolated laboratory environment using Kali Linux and TShark.

Tools Used

Tool	Purpose
Kali Linux	Digital-forensics analysis environment
TShark	Command-line packet analysis
Wireshark	Packet-level visual verification
sha256sum	Evidence integrity verification
Apache HTTP Server	TCP/HTTP service used for baseline activity
PCAP file	Offline network evidence

Evidence File

Filename: mySYNFloodCapture.pcap

Location: evidence/mySYNFloodCapture.pcap

The original capture was preserved and analysed without modifying the evidence.

4. Authorisation and Safety

All traffic analysis was performed for the authorised SBT-DF203 laboratory exercise. The investigation was restricted to the supplied training capture and the controlled local/loopback environment.

No third-party, production, public, or external network was targeted. The analysis was performed as a forensic examination of captured traffic rather than an attempt to disrupt a live service.

5. TCP Three-Way Handshake Baseline

A normal TCP connection is established using a three-way handshake:

1. **SYN** — The client requests a TCP connection.
2. **SYN-ACK** — The server acknowledges the request and responds with its own SYN.
3. **ACK** — The client acknowledges the server response.

The expected sequence is therefore:

Client → Server: SYN

Server → Client: SYN-ACK

Client → Server: ACK

When this sequence completes, the TCP connection can proceed to application-layer communication such as HTTP.

In contrast, a capture containing repeated initial SYN packets without corresponding SYN-ACK and ACK packets can indicate incomplete connection attempts. Such a pattern can be relevant to SYN-flood investigations, although packet evidence alone does not establish that a denial-of-service condition occurred.

File information:

file evidence/"mySYNFloodCapture (3).pcap"

capinfos evidence/"mySYNFloodCapture (3).pcap"

```
(kali@kali)-[~/icdfaDF/STB-DF203-Lab3]
$ file evidence/"mySYNFloodCapture.pcap"
capinfos evidence/"mySYNFloodCapture.pcap"
evidence/mySYNFloodCapture.pcap: pcapng capture file - version 1.0
File name:          evidence/mySYNFloodCapture.pcap
File type:          Wireshark/... - pcapng
File encapsulation: Ethernet
File timestamp precision: nanoseconds (9)
Packet size limit:  file hdr: (not set)
Number of packets:  4
File size:          260 kB
Data size:          260 kB
Capture duration:   0.002756569 seconds
Earliest packet time: 2023-02-06 11:36:37.366389062
Latest packet time:  2023-02-06 11:36:37.369145631
Data byte rate:     94 MBps
Data bit rate:      755 Mbps
Average packet size: 65054.00 bytes
Average packet rate: 1,451 packets/s
SHA256:             14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5
SHA1:               0fc44e40c26068511571b53eebc5641de82eea90
Strict time order:  True
Capture hardware:   AMD Ryzen Threadripper 3970X 32-Core Processor (with SSE4.2)
Capture oper-sys:   Linux 5.14.0-kali4-amd64
Capture application: Dumpcap (Wireshark) 3.4.9 (Git v3.4.9 packaged as 3.4.9-1)
Number of interfaces in file: 1
Interface #0 info:
    Name = lo
    Encapsulation = Ethernet (1 - ether)
    Capture length = 262144
    Time precision = nanoseconds (9)
    Time ticks per second = 1000000000
    Time resolution = 0x09
    Operating system = Linux 5.14.0-kali4-amd64
    Number of stat entries = 1
    Number of packets = 4
```

Check that TShark works

tshark --version

```

(kali㉿kali)-[~/icdfaDF/STB-DF203-Lab3]
$ tshark --version
TShark (Wireshark) 4.6.6.

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Compile-time info:
  Bit width: 64-bit
  Compiler: GCC 15.2.0
  Glib: 2.88.1
With:
  +brotli
  +Gcrypt 1.12.2
  +GnuTLS 3.8.13 and PKCS#11
  +Kerberos (MIT)
  +libnl 3
  +libpcap
  +libsmb 0.4.8
  +libxml2 2.15.2
  +Lua 5.4.8
  +lz4 1.10.0
  +MaxMind
  +nghttp2 1.69.0
  +nghttp3 1.15.0
  +PCRE2 10.46 2025-08-27
  +POSIX capabilities (Linux)
  +Snappy 1.2.2
  +xxhash 0.8.3
  +zlib 1.3.2
  +Zstandard 1.5.7

```

Then run:

```
tshark -r evidence/"mySYNFloodCapture.pcap" -c 10
```

```

(kali㉿kali)-[~/icdfaDF/STB-DF203-Lab3]
$ tshark -r evidence/"mySYNFloodCapture.pcap" -c 10
  1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 65054 62558 → 80 [SYN] Seq=0 Win=8192 Len=65000
  2 0.000899410 127.0.0.1 → 127.0.0.1 TCP 65054 19550 → 80 [SYN] Seq=0 Win=8192 Len=65000
  3 0.001784646 127.0.0.1 → 127.0.0.1 TCP 65054 63314 → 80 [SYN] Seq=0 Win=8192 Len=65000
  4 0.002756569 127.0.0.1 → 127.0.0.1 TCP 65054 34942 → 80 [SYN] Seq=0 Win=8192 Len=65000

(kali㉿kali)-[~/icdfaDF/STB-DF203-Lab3]
$

```

6. Capture Examination

The supplied PCAP was examined using TShark.

The following command was used to extract TCP packet information:

```

tshark -r evidence/"mySYNFloodCapture.pcap" \
-Y tcp \
-T fields \

```

```
-E separator='|' \  
-e frame.number \  
-e frame.time_relative \  
-e ip.src \  
-e tcp.srcport \  
-e ip.dst \  
-e tcp.dstport \  
-e tcp.flags.str
```

The resulting packet records were:

```
1|0.000000000|127.0.0.1|62558|127.0.0.1|80|.....S.  
2|0.000899410|127.0.0.1|19550|127.0.0.1|80|.....S.  
3|0.001784646|127.0.0.1|63314|127.0.0.1|80|.....S.  
4|0.002756569|127.0.0.1|34942|127.0.0.1|80|.....S.
```

```
(kali㉿kali) [~/icdfadF/ST-DF203-Lab3]  
$ tshark -r evidence/"mySYNFloodCapture.pcap" \  
-T fields \  
-E header=y \  
-E separator='|' \  
-e frame.number \  
-e frame.time_relative \  
-e ip.src \  
-e tcp.srcport \  
-e ip.dst \  
-e tcp.dstport \  
-e tcp.flags.str \  
-e tcp.seq \  
-e tcp.ack \  
  
frame.number|frame.time_relative|ip.src|tcp.srcport|ip.dst|tcp.dstport|tcp.flags.str|tcp.seq|tcp.ack  
1|0.000000000|127.0.0.1|62558|127.0.0.1|80|.....S.|0|0  
2|0.000899410|127.0.0.1|19550|127.0.0.1|80|.....S.|0|0  
3|0.001784646|127.0.0.1|63314|127.0.0.1|80|.....S.|0|0  
4|0.002756569|127.0.0.1|34942|127.0.0.1|80|.....S.|0|0
```

7. Quantitative Analysis

7.1 Total TCP Packets

The total number of TCP packets was calculated using:

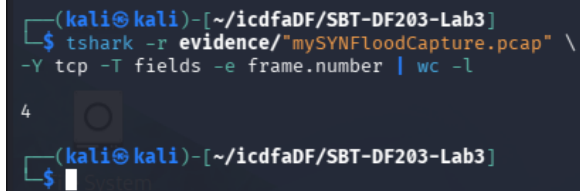
```
tshark -r evidence/"mySYNFloodCapture.pcap" \  
-Y tcp -T fields -e frame.number | wc -l
```

Result:

4

Therefore:

Total TCP packets = 4



```
(kali㉿kali)-[~/icdfaDF/GBT-DF203-Lab3]
$ tshark -r evidence/"mySYNFloodCapture.pcap" \
-Y tcp -T fields -e frame.number | wc -l
4
(kali㉿kali)-[~/icdfaDF/GBT-DF203-Lab3]
$
```

7.2 SYN Packet Count

Initial SYN packets were identified using:

```
tshark -r evidence/"mySYNFloodCapture.pcap" \
-Y "tcp.flags.syn == 1 && tcp.flags.ack == 0" \
-T fields \
-E separator='|' \
-e frame.number \
-e frame.time_relative \
-e ip.src \
-e tcp.srcport \
-e ip.dst \
-e tcp.dstport \
-e tcp.flags.str
```

The command returned four packets.

Therefore:

Initial SYN count = 4

```
(kali@kali)-[~/icdfaDF/GBT-DF203-Lab3]
$ tshark -r evidence/"mySYNFloodCapture.pcap" \
-Y "tcp.flags.syn = 1 && tcp.flags.ack = 0" \
-T fields \
-E separator='|' \
-e frame.number \
-e frame.time_relative \
-e ip.src \
-e tcp.srcport \
-e ip.dst \
-e tcp.dstport \
-e tcp.flags.str
1|0.000000000|127.0.0.1|62558|127.0.0.1|80|.....S.
2|0.000899410|127.0.0.1|19550|127.0.0.1|80|.....S.
3|0.001784646|127.0.0.1|63314|127.0.0.1|80|.....S.
4|0.002756569|127.0.0.1|34942|127.0.0.1|80|.....S.
```

7.3 SYN-ACK Packets

The capture was examined for SYN-ACK responses.

Observed SYN-ACK packets = 0

No SYN-ACK response was present in the supplied capture.

```
(kali@kali)-[~/icdfaDF/GBT-DF203-Lab3]
$ tshark -r evidence/"mySYNFloodCapture.pcap" \
-Y "tcp.flags.syn = 1 && tcp.flags.ack = 1" \
-T fields \
-e frame.number \
-e frame.time_relative \
-e ip.src \
-e tcp.srcport \
-e ip.dst \
-e tcp.dstport \
-e tcp.flags.str
(kali@kali)-[~/icdfaDF/GBT-DF203-Lab3]
$
```

7.4 ACK Packets

The capture was examined for TCP ACK packets associated with completed handshakes.

Observed ACK packets = 0

No completing ACK packet was observed.

```
(kali@kali)-[~/icdfaDF/GBT-DF203-Lab3]
$ tshark -r evidence/"mySYNFloodCapture.pcap" \
-Y "tcp.flags.ack = 1 && tcp.flags.syn = 0" \
-T fields \
-e frame.number \
-e frame.time_relative \
-e ip.src \
-e tcp.srcport \
-e ip.dst \
-e tcp.dstport \
-e tcp.flags.str
```

7.5 RST Packets

The capture was also examined for TCP reset packets.

Observed RST packets = 0

No RST packet was observed.

```
(kali@kali)-[~/icdfaDF/GBT-DF203-Lab3]
$ tshark -r evidence/"mySYNFloodCapture.pcap" \
-Y "tcp.flags.reset = 1" \
-T fields \
-e frame.number \
-e frame.time_relative \
-e ip.src \
-e tcp.srcport \
-e ip.dst \
-e tcp.dstport \
-e tcp.flags.str
```

8. TCP Flag Distribution

The TCP flag distribution was examined using:

```
tshark -r evidence/"mySYNFloodCapture.pcap" \
-Y tcp \
-T fields \
-e tcp.flags.str | sort | uniq -c
```

Result:

4S·

This shows that all four TCP packets contained the SYN flag pattern.

```
(kali@kali)-[~/icdfaDF/GBT-DF203-Lab3]
$ tshark -r evidence/"mySYNFloodCapture.pcap" \
-Y tcp \
-T fields \
-e tcp.flags.str | sort | uniq -c

4 .....S.
```

Summary

TCP Packet Type	Count
SYN	4
SYN-ACK	0
ACK	0
RST	0
Total TCP packets	4

The observed TCP traffic therefore consisted entirely of initial SYN packets.

9. Source Port Analysis

The four observed source ports were:

Frame	Source IP	Source Port	Destination IP	Destination Port
1	127.0.0.1	62558	127.0.0.1	80
2	127.0.0.1	19550	127.0.0.1	80
3	127.0.0.1	63314	127.0.0.1	80
4	127.0.0.1	34942	127.0.0.1	80

Findings

- Total SYN packets: **4**
- Unique source ports: **4**
- Source IP: **127.0.0.1**
- Destination IP: **127.0.0.1**
- Destination TCP port: **80**

The use of different source ports means that each packet represents a distinct TCP connection tuple rather than repeated packets from one identical source port.

10. Forensic Timeline

Relative Time	Frame	Source	Destination	Event	Interpretation
0.000000000s	1	127.0.0.1:62558	127.0.0.1:80	SYN	Initial connection attempt
0.000899410s	2	127.0.0.1:19550	127.0.0.1:80	SYN	Second connection attempt
0.001784646s	3	127.0.0.1:63314	127.0.0.1:80	SYN	Third connection attempt
0.002756569s	4	127.0.0.1:34942	127.0.0.1:80	SYN	Fourth connection attempt

The first packet occurred at 0.000000000 seconds and the final packet occurred at 0.002756569 seconds.

Therefore, the four observed SYN packets occurred within approximately:

0.002756569 seconds = 2.76 milliseconds

This demonstrates that the four SYN packets occurred within a very short interval in the supplied training capture.

11. Incomplete Handshake Analysis

A normal TCP handshake requires:

SYN → SYN-ACK → ACK

The supplied capture instead contains:

SYN
SYN
SYN
SYN

No SYN-ACK or ACK packets were observed.

Consequently, the supplied capture contains **four observed initial connection attempts without corresponding handshake-completion packets within the captured traffic.**

This is an indicator of incomplete TCP handshakes.

However, the absence of responses in this particular capture does not necessarily mean that the server failed to respond. The capture may be intentionally bounded and may not contain the complete network conversation. Therefore, the correct forensic conclusion is that **no corresponding SYN-ACK, ACK, or RST packets are present in the examined evidence**, rather than claiming that the server definitely failed to respond.

12. Normal Versus Suspicious Indicators

Indicator	Normal TCP Connection	Supplied Training Capture
Initial SYN	Present	Present
SYN-ACK	Expected	0 observed
Final ACK	Expected	0 observed
RST	Possible in some conditions	0 observed
Handshake completion	Expected	Not observed
Source ports	Usually connection-specific	4 unique ports
Destination	Service endpoint	127.0.0.1:80
Traffic duration	Depends on connection	~2.76 ms

Evidence of service impact	Requires additional evidence	Not established
----------------------------	------------------------------	-----------------

The main suspicious indicator is the presence of multiple SYN packets without corresponding handshake-completion packets.

13. Evidence Integrity

A SHA-256 hash was calculated for the original PCAP to support evidence integrity.

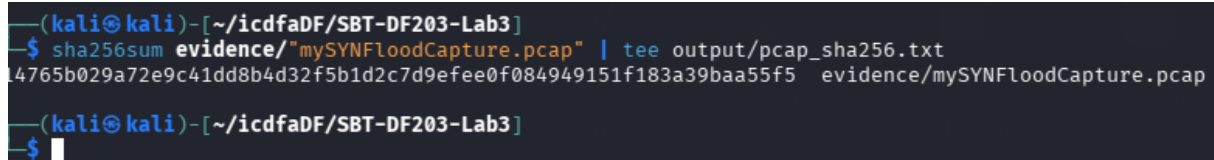
Command used:

```
sha256sum evidence/"mySYNFloodCapture.pcap" \  
> output/pcap_sha256.txt
```

SHA-256

14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5
evidence/mySYNFloodCapture.pcap

The original evidence file should remain preserved, while analysis should be performed on a verified working copy where required by the laboratory procedure.



```
(kali㉿kali)-[~/icdfaDF/GBT-DF203-Lab3]  
$ sha256sum evidence/"mySYNFloodCapture.pcap" | tee output/pcap_sha256.txt  
14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5  evidence/mySYNFloodCapture.pcap  
  
(kali㉿kali)-[~/icdfaDF/GBT-DF203-Lab3]  
$
```

14. TShark Evidence Outputs

The following extraction files were generated during analysis:

Initial SYN extraction

output/initial_syms.txt

Complete TCP analysis

output/tcp_packets.txt

PCAP SHA-256 record

output/pcap_sha256.txt

These files provide reproducible command-line evidence supporting the packet counts and observations documented in this report.

```
(kali㉿kali)-[~/icdfaDF/SBT-DF203-Lab3]
$ tree
.
├── analysis
├── evidence
│   └── mySYNFloodCapture.pcap
├── output
│   ├── initial_syns.txt
│   ├── pcap_sha256.txt
│   └── tcp_packets.txt
└── screenshots

5 directories, 4 files
```

15. Interpretation of Findings

The supplied capture demonstrates a bounded pattern of repeated TCP SYN activity directed at TCP port 80.

Four SYN packets were observed from 127.0.0.1 to 127.0.0.1:80, each using a different source port. No SYN-ACK, ACK, or RST packets were present in the capture.

From a forensic perspective, this is consistent with incomplete TCP connection attempts.

The packets occurred within approximately 2.76 milliseconds, demonstrating that the four connection attempts were closely grouped in time.

However, the evidence must be interpreted within the limitations of the supplied capture. Four packets are insufficient to establish a sustained SYN flood or demonstrate that the Apache service experienced denial of service.

16. What the Evidence Does Not Prove

The capture alone does **not** prove:

1. That Apache became unavailable.
2. That system resources were exhausted.
3. That the TCP connection backlog was exhausted.
4. That legitimate users experienced degraded service.
5. That a sustained high-volume attack occurred.
6. That the server ignored the SYN packets.
7. That the traffic originated from an external attacker.
8. That a denial-of-service condition occurred.

The source and destination address 127.0.0.1 instead place the observed traffic within the local loopback environment represented by the training capture.

17. Additional Evidence Required to Establish Service Impact

To determine whether the SYN activity caused an actual service impact, additional evidence would be useful, including:

- Longer-duration packet captures.
- Server availability measurements.
- Apache access and error logs.
- TCP connection and backlog statistics.
- CPU and memory utilisation.
- Network-interface statistics.
- Application response-time measurements.
- Evidence of legitimate connection failures.
- Comparison with normal baseline traffic.
- Host-level event or system logs.

Correlating these sources would provide stronger evidence of whether abnormal SYN activity resulted in an actual availability problem.

18. Detection Recommendations

The following controls can help detect similar activity:

18.1 Packet Monitoring

Monitor for unusually high numbers of initial SYN packets over a defined period.

18.2 SYN/SYN-ACK Ratio

Compare the number of incoming SYN packets with corresponding SYN-ACK and completed ACK packets. A large imbalance can be an indicator requiring investigation.

18.3 Source-Port Analysis

Track repeated connection attempts and the number of unique source ports associated with a destination service.

18.4 IDS/IPS Monitoring

Network intrusion-detection systems can alert on abnormal TCP handshake patterns.

18.5 Server Monitoring

Monitor TCP connection states, backlog utilisation, CPU usage, memory usage, and application availability.

18.6 Log Correlation

Correlate network evidence with Apache logs and operating-system events to establish whether suspicious traffic affected the application.

19. Mitigation Recommendations

Appropriate defensive measures include:

1. Configure suitable firewall and network filtering policies.
2. Monitor TCP connection rates.
3. Apply operating-system TCP hardening appropriate to the environment.
4. Use rate limiting where appropriate.
5. Deploy IDS/IPS monitoring.
6. Monitor server resource utilisation.
7. Use upstream network protection where a real service is exposed to large-scale traffic.
8. Maintain accurate network and application logs.
9. Establish alert thresholds based on a known normal baseline.
10. Regularly review firewall and server configurations.

Mitigation should be selected according to the actual environment and should not interfere with legitimate traffic.

20. Screenshots and Evidence Checkpoints

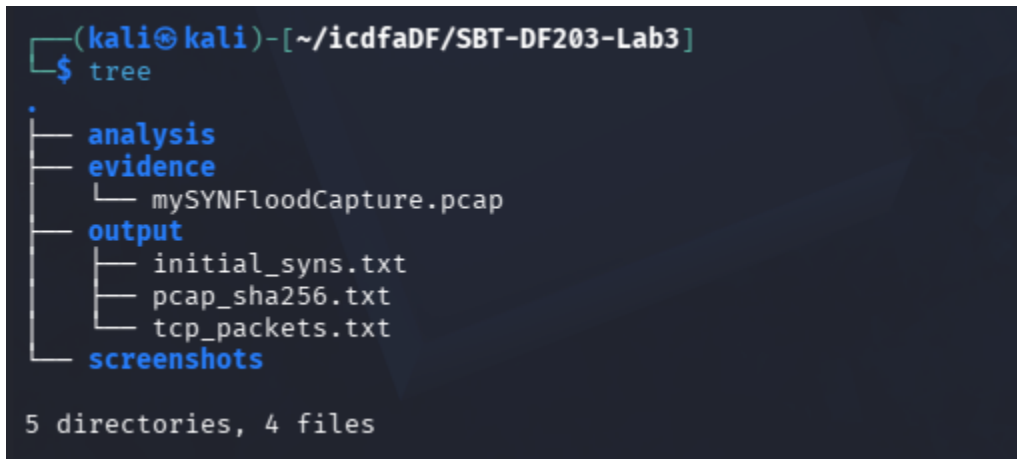
The following screenshots should be included in the final submission and referenced in the report.

Screenshot	Evidence
Figure 1	Kali laboratory environment
Figure 2	PCAP file location
Figure 3	TShark TCP extraction
Figure 4	Initial SYN filter
Figure 5	SYN-ACK filter showing no matching packets
Figure 6	ACK filter showing no matching packets
Figure 7	RST filter showing no matching packets
Figure 8	TCP flag distribution
Figure 9	Source-port analysis
Figure 10	SHA-256 hash calculation

All screenshots should be readable, numbered, and correspond to commands or evidence actually produced during the investigation.

21. Evidence Directory

The final evidence directory should contain the relevant laboratory material, for example:



```
(kali㉿kali)-[~/icdfaDF/SBT-DF203-Lab3]
$ tree
.
├── analysis
├── evidence
│   └── mySYNFloodCapture.pcap
├── output
│   ├── initial_syns.txt
│   ├── pcap_sha256.txt
│   └── tcp_packets.txt
└── screenshots

5 directories, 4 files
```

22. Conclusion

This investigation successfully demonstrated the use of TShark to identify and quantify TCP SYN activity in an authorised digital-forensics training environment.

The supplied PCAP contained four TCP packets. All four were initial SYN packets directed from 127.0.0.1 to TCP port 80, using four unique source ports: 62558, 19550, 63314, and 34942. No SYN-ACK, ACK, or RST packets were observed.

The packets occurred within approximately 2.76 milliseconds, producing a clear bounded pattern of incomplete TCP connection attempts in the examined capture.

The findings demonstrate how packet captures can be used to identify potential SYN-related anomalies and construct a forensic timeline. Nevertheless, the evidence is insufficient on its own to establish that a SYN flood caused denial of service. Additional host, application, network, and availability evidence would be necessary to demonstrate actual service impact.

The investigation therefore concludes that the supplied capture provides **evidence of repeated initial SYN activity and incomplete handshakes within the captured dataset, but does not independently prove a successful denial-of-service event.**